Ders 10 Genişletilmiş Notu: Yazılım Kalitesi, Test Stratejisi ve Test Türleri

Ders 10 Genişletilmiş Notu: Yazılım Kalitesi, Test Stratejisi ve Test Türleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Yeni Bölüm: Kalite ve Test
- 2. Süreç Adımları İçinde Test
- 3. Kodlama Aşamasında Kalite
- 4. Yazılım Kalite Faktörleri
- 5. Kalite Modelleri: ISO Standardı ve McCall Modeli
- 6. Kaliteyi Sağlamak ve Hataları Erken Bulmak
- 7. Gözden Geçirme, İnceleme ve Test
- 8. Test Stratejisi: Test Adımları
- 9. Test Tipleri: Saydam Kutu, Kara Kutu, Gri Kutu
- 10. Kara Kutu Testinde Test Verisi Üretmek
- 11. Test Otomasyonu
- 12. Doğrulama ve Geçerleme
- 13. Test ve Hata Ayıklama
- Örnek Sorular
- Dersin Toparlaması

1. Yeni Bölüm: Kalite ve Test

Hoca derse, önceki haftalarda nesneye yönelik analiz ve tasarımla ilgili diyagramların ve uygulamaların tamamlandığını hatırlatarak başladı. Gelecek hafta ders asistanı bu konuda bir uygulama, bir vaka (case) çalışması yaptıracaktı. Bu hafta yeni bir konuya geçildi: yazılım kalitesi, kalite özellikleri ve ardından test. Sonraki derste de yazılım proje yönetimi ve ölçümü işlenecekti.

2. Süreç Adımları İçinde Test

Hoca önce yazılım sürecinin adımlarını bir kez daha gözden geçirdi, çünkü testin bu süreçte nerede durduğunu görmek gerekiyordu:

- Sistem gereksinimleri analizden alınır.
- Tasarım gerçekleştirilir ve tasarımın değerlendirmesi yapılır.
- Kodlama yapılır, programlar ortaya çıkar.
- Programlar üzerinde önce sözdizimiyle (syntax) ilgili değerlendirme yapılır.
- Test aşamasına geçilir. Önce test planına göre işlemler yürütülür ve modüller test edilir.
- Genel sistem testi yapılır.
- Son olarak kullanıcı onay testi yapılır. Plan ve yapılan test işlemleri kaydedilerek sistem kullanıma hazır hâle getirilir.

3. Kodlama Aşamasında Kalite

Kodlama aşamasında programlama dili seçilir ve kod yazılır. Hoca bu aşamanın ayrıntısına bu derste girmedi, çünkü kodlama ve programlama kuralları programlama derslerinde zaten görülüyor. Yine de kaliteyle ilgili birkaç noktanın altını çizdi.

3.1 Programlama Dili Seçimi

Hangi tür yazılım için hangi programlama dilinin uygun olduğuna gereksinim analizi aşamasında karar verilir. Bazen sistemin farklı yerlerinde birden fazla dili devreye sokmak gerekir. Dil seçiminde bakılan ölçütler şunlardır:

- Genel uygulama alanı: hangi alanda çalışıyoruz?
- Algoritmanın karmaşıklık derecesi.
- Yazılımcının kullanacağı ortam.
- Uygulama koşulları.
- Çalışanların bilgi düzeyi: hangi dili daha iyi biliyorlar?
- Proje koşullarıyla ilgili başka pek çok faktör.

3.2 Açıklamalar, Adlandırma ve Etkin Kod

Kodu yazarken yazılımın ve değişkenlerin anlaşılmasını sağlayan açıklamalar koymak gerekir. Hoca bunu pek sevmediğimizi, vakit alıyormuş gibi göründüğünü söyledi. Ama bu açıklamalar bakım aşamasında işe yarayacak bilgileri içerir. Değişkenlere anlamlı adlar vermek de daha uygundur. Bunun yanında algoritmalar, programlama dilleri ve giriş derslerinde görüldüğü gibi etkin (verimli) programlamayı sağlayacak şekilde kod yazılmalıdır.

4. Yazılım Kalite Faktörleri

Hoca “kaliteli bir yazılım dediğimizde neyi anlıyoruz?” sorusuyla kalite faktörlerine geçti. Kaliteli yazılımı belirten özellikler şunlardır: doğruluk, güvenilirlik, verimlilik, güvenlik, kullanışlılık, hata bulma kolaylığı, esneklik, sınama kolaylığı, taşınabilirlik, tekrar kullanılabilirlik ve birlikte çalışabilirlik. Bu terimlerin İngilizce karşılıkları daha sık duyulduğu için hoca her birini İngilizcesiyle birlikte açıkladı.

4.1 Doğruluk, Güvenilirlik, Verimlilik, Güvenlik

- **Doğruluk (correctness):** Yazılımın doğru çalışması, spesifikasyonlara uygunluğu. Müşteri isteklerini ne kadar iyi karşıladığını anlatır.
- **Güvenilirlik (reliability):** Yazılım tasarlanan işlevleri istenen duyarlılıkla yerine getiriyor mu?
- **Verimlilik (efficiency):** Program işlevlerini yerine getirirken bilgi işlem kaynaklarını ne kadar verimli kullanıyor? Hoca buna zaman zaman etkinlik de dendiğini söyledi.
- **Güvenlik (security):** Yetkisiz kişilerin yazılıma müdahale etmesinin önlenmesi.

4.2 Kullanışlılık

Kullanışlılık (usability): Yazılımı öğrenmek, işletmek, girdi hazırlamak ve çıktıları yorumlamak ne kadar kolay? Hoca bunun kullanıcı dostu (user friendly) olmak diye de tanımlandığını, yani yazılımın kullanıcıya ne kadar dost olduğunu anlattığını söyledi.

4.3 Bakımcı Gözüyle Faktörler

Hata bulma kolaylığı, esneklik ve sınama kolaylığı özellikle bakım yapacak kişiler için çok önemlidir. Değişiklik yapmak, hatayı bulmak ve düzeltmek ne kadar kolay? **Sınama kolaylığı (testability)**, yazılımın test edilmesinin ne kadar kolay olduğunu belirtir.

4.4 Taşınabilirlik

Taşınabilirlik (portability): Yazılımın bir donanım biriminden başka bir donanıma ya da başka bir yazılım sistemi ortamına aktarılabilmesi.

4.5 Tekrar Kullanılabilirlik

Tekrar kullanılabilirlik (reusability): Yazılımın tamamının ya da bir bölümünün farklı bir uygulamada kullanılabilmesi. Hoca bu kavramın tasarım konusunda ele alındığını ve son yıllarda yazılım mühendisliğinde çok gündemde olduğunu söyledi. Önemi, aynı eforun tekrar harcanmamasıdır.

Burada hoca sınıfa, tasarım konusunda görülen kavramlardan hangilerinin yeniden kullanılabilirliği sağladığını sordu. Bu soru ve cevapları "Örnek Sorular" bölümünde ayrıca verildi. Toparlanan cevaplar şunlardır:

- **Nesneye yönelik yaklaşım:** Yeniden kullanılabilirliği artırır.
- **Az bağımlılık ve modülerlik:** Modülerlik az bağımlılığı sağlar. Sınıflar arasındaki bağımlılığın az olması da aynı amaca hizmet eder.
- **Tasarım kalıpları (design patterns):** Tamamen yeniden kullanılabilirlikle ilgilidir. Hocanın ifadesiyle: "Çözdüğünüz bir problemi tekrar çözmekle uğraşmayın, tasarım kalıplarını kullanın."
- **Yazılım kütüphaneleri:** Önceden hazırlanmış fonksiyonların kütüphanelere eklenmesi ve kullanılması.

Bunlar yazılım hazırlamadaki verimliliği artırır ve daha az iş gücü kullanılmasını sağlar. Ayrıca doğru olarak test edilmiş parçalar kullanıldığı için test etme işini de kolaylaştırır.

4.6 Birlikte Çalışabilirlik

Birlikte çalışabilirlik (interoperability): Başka yazılımlara veri aktarma, onlardan veri alma ve onlarla birlikte çalışma özelliği. Hocanın slaytında bu özellik "bağlanabilirlik" diye geçiyordu. Hoca, daha uzun bir ifade olsa da "birlikte çalışabilirlik" teriminin interoperability'nin karşılığını daha iyi anlattığını söyledi.

5. Kalite Modelleri: ISO Standardı ve McCall Modeli

Kaliteli yazılımı en iyi anlatan bazı standartlar ve modeller vardır. Hoca bunlardan ikisini andı.

5.1 ISO Standardı

Birincisi bir ISO standardıdır. Bu standart kalite faktörlerini sayarak (doğruluk, güvenilirlik, verimlilik, test etme kolaylığı gibi) kaliteli yazılımın nasıl olduğunu belirtir.

> **Ders sonrası not:** Altyazıda standardın numarası "ISO 9176" diye geçiyor. Yazılım ürün kalitesi modeli olarak bilinen standart büyük olasılıkla **ISO/IEC 9126**'dır. Bu standardın yerini büyük olasılıkla 2011'de **ISO/IEC 25010** almıştır.

5.2 McCall Modeli

En çok kullanılan model McCall modelidir. Bu modelde kalite faktörleri doğrudan ölçülmez. Her faktör, bazı özelliklere bağlı bir doğrusal regresyon denklemiyle tahmin edilir.

Hoca bunu sorularla açtı. Taşınabilirlik diyoruz, ama bunu neye bağlı olarak söylüyoruz? Güvenilirlik diyoruz, ama güvenilirlik nelere bağlı, neyle ölçülür, hangi metrikler güvenilirliği en çok destekler? Doğruluk nelere bağlıdır? Örneğin hata sayısı, hata bulma kolaylığı gibi ölçütler. McCall modelinde her yazılım kalite faktörü, metriklerin belirli katsayılarla ağırlıklandırılmış toplamı olarak ifade edilir:

$$F_q = c_1 m_1 + c_2 m_2 + \cdots + c_n m_n$$

Burada F_q kalite faktörü, m_i metrikler, c_i katsayılardır. Böylece bir kalite faktörü birden fazla metrik cinsinden belirtilebilir. Hoca, yazılım ölçütleri (software metrics) konusunun yazılım proje yönetimi ve ölçümü içinde, bir sonraki haftadan sonraki derste işleneceğini söyledi.

> **Ders sonrası not:** Altyazıda modelin adı “MacCol” ve “McColl” diye de geçiyor. Doğrusu McCall’dır. Model büyük olasılıkla Jim McCall ve arkadaşlarının 1977 tarihli çalışmasına dayanır.

6. Kaliteyi Sağlamak ve Hataları Erken Bulmak

6.1 Kaliteyi Güvence Altına Alan İşlemler

Hoca, yazılım geliştirme sırasında daha kaliteli yazılım için yapılanları şöyle sıraladı:

- Gereksinim analizini daha iyi yapmak ve daha planlı geliştirmek.
- Modüllere ayırırken ve tasarım sırasında taşınabilirlik ve birlikte çalışabilirlik özelliklerine önem vermek. Modülleri ve sınıfları belirlerken tasarımı daha iyi yapmak.
- Ayrıntılı tasarımda, yani girdi, çıktı ve veritabanı tasarımında, doğruluk faktörüne ağırlık vermek ve kullanıcının gereksinimlerine uygun hareket etmek.
- Test aşamasında performansı ve diğer faktörleri ölçmeye çaba göstermek.

6.2 Hata Bulma Maliyeti

Kaliteyi sağlamak için dikkat edilecek ilk şey, hataları olabildiğince erken bulmaya çalışmaktır. Hoca bunu ilk derslerde ve sistem analizi dersinde de çizdiği grafiklerle hatırlattı:

- Süreç ilerledikçe hata bulma gideri artar.
- Süreç ilerledikçe hata bulma olasılığı düşer.

Geç evrelerde hata bulmak daha zordur, daha çok efor ve iş gücü ister, bu yüzden daha çok gidere yol açar. Ayrıca bir basamakta yapılan hata sonraki basamaklara taşınır.

> **Hocanın vurgusu:** Bu diyagramları hep gözünüzün önünde bulundurun. “İlk aşamaları çok hızlı geçelim, bir an önce kodlamaya geçelim, yazılım kullanılmaya başlasın” demek, sonradan daha büyük ve daha zor bulunan hatalara yol açabilir.

7. Gözden Geçirme, İnceleme ve Test

Yazılım geliştirme sürecinde hata aramak ve kalite kontrolü yapmak için bir inceleme ve kontrol planı uygulanır. Kaliteyi sağlamak için yapılan işlemlerden biri yazılım testidir (yazılım sınama). Ama testten önce yapılan iki işlem daha vardır: yazılımın gözden geçirilmesi (review) ve yazılımın incelenmesi (software inspection). Bunların ardından yazılım test edilir.

7.1 Testin Tanımı

Yazılım testi, bir programdaki hataları bulmak amacıyla yapılan işlemdir. Testin işi yalnızca hatayı bulmaktır. Hatayı düzeltmek bunun dışındadır ve hata ayıklama (debug) işlemiyle yapılır. Test, neyi sınadığına göre fonksiyonel sınama, performans sınaması gibi adlar alabilir. Yapısal olarak dayanıklılık, performans ve fonksiyonel olarak yapılan işin doğru iş olup olmadığı test edilir.

Test işlemleri daha çok kara kutu testi ve saydam kutu testi olarak görülür. Bu test tipleri gri kutu testiyle de desteklenir. Hoca test tiplerine geçmeden önce testin hangi sırayla yapılacağını, yani test stratejisini anlattı.

> **Ek bilgi (derste anlatılmadı):** Gözden geçirme (review) genelde bir belge ya da kod parçasının ekip içinde daha serbest biçimde değerlendirilmesidir. İnceleme (inspection) ise büyük olasılıkla Fagan’ın önerdiği türden, rolleri (moderatör, okuyucu, yazar, kayıtçı) ve kontrol listeleri tanımlı, daha resmî bir süreçtir. İkisi de kodu çalıştırmadan yapılan statik denetimlerdir; test ise kodu çalıştırır.

8. Test Stratejisi: Test Adımları

Ara sonrasında hoca test sürecine geçti. Test süreci, yazılım geliştirme süreciyle ters yönde ilişkilidir. Geliştirirken sistem gereksinimlerinden tasarıma, tasarımdan kodlamaya iniyorsak, testte kodlamanın hemen arkasından birim testi, sonra bütünleştirme testi, sonra sistem testi ve en son onay (kabul) testi gelir.

```
Geliştirme (aşağı iner)          Test (yukarı çıkar)
Kullanıcı / sistem gereksinimleri  <->  Sistem testi, kabul testi
Tasarım                            <->  Bütünleştirme testi
Kodlama                            <->  Birim testi
```

8.1 Birim Testi

Birim testinde, kodlamadan hemen sonra yazılan birim test edilir. Bunun için kodun açık olması, yani görülebilmesi gerekir. Modülün algoritması ve modüldeki yapılar test edilir. Bu yüzden burada saydam kutu testi yapılır. Birim testi geliştirmeyle paralel yürütülebilir: bir kod, bir modül oluşturuldukça onun birim testi yapılır.

Birim testinde şunlara bakılır:

- **Modülün ara birimi (arayüzü):** Verinin girişi, çıkışı ve uygunluğu.
- **Veri yapısı:** Değişkenlerin durumu, veri tipinin uygunluğu, kullanılan veri yapısı.
- **Kontrol yapıları:** Kontrol akışının nasıl gerçekleştiği ve karşılaştırmalar.
- **Ana yollar:** Başlangıçtan sona kadar bütün algoritma yolları.
- **Hata arama yolları.**
- **Modül sınırları.**

Her modül algoritma adımlarına uygun biçimde test edilir. Hedef, başlangıçtan sona ulaşana kadar bütün algoritma yollarının test edilmesidir. Kaç tane temel yol varsa hepsinin test edilmesi hedeflenir. Bunun için farklı değişkenlere farklı girdiler verilerek farklı veri gruplarıyla test yapılır. Modül sınırlarında daha çok hata çıkacağı tahmin edildiği için test özellikle sınır değerlerde yoğunlaşır. Ağırlıklı olarak saydam kutu testi (white box testing) kullanılır, zaman zaman kara kutu testi de yapılabilir.

8.2 Bütünleştirme Testi

Modüller bağımsız değildir, birbirine bağlıdır. Bu bağlantının fazla olmaması istenir; hoca bunun reusability sorusunda bir öğrencinin söylediği az bağımlılıkla ilgili olduğunu hatırlattı. Bütünleştirme testinde ara birimler ve modüllerin birbirleriyle ilişkileri kontrol edilir; bu ilişkilerde hata olmaması gerekir.

Ana program, modüllerin birleştirilmesiyle oluşur. Bütünleştirme testi iki şekilde yapılabilir: ana programı bir bütün olarak test etmek ya da artırmalı (artımlı) olarak test etmek.

- **Big bang testi:** Modüller rastgele, bir kerede birleştirilerek test edilir.
- **Artırmalı sınama, aşağıdan yukarıya:** Alttaki modüller birleştirilir ve kümeler (cluster) oluşturularak yukarıya doğru birleştirilir.
- **Artırmalı sınama, yukarıdan aşağıya:** Üstteki modülden başlanarak aşağıya doğru ilerlenir.
- **Sandviç testi:** Her zaman tek yönde gidilmez. Modüllerin bir kısmı aşağıdan yukarıya bağlanırken arada yukarıdan aşağıya da test yapılır. Bu, daha sağlıklı bir test sağlar.

  Ders sonrası not: Hoca bu adımı altyazıda çoğunlukla “bütünlük testi” diye anıyor; dersin sonunda bir öğrenci İngilizcesini “integrity test” dedi ve hoca itiraz etmedi. Doğru terim **integration testing** (bütünleştirme, tümleştirme testi) olmalıdır. Integrity, bütünlük ya da veri tutarlılığı anlamına gelir.

8.3 Sistem Testi

Yazılım kendi içinde birim ve bütünleştirme testleriyle doğru çalışıyor olarak tanımlandı. Sistem testinde, kendi içinde doğru çalışan bu yazılımın donanım ve yazılım altyapısı üzerinde nasıl çalıştığı belirlenir: veritabanı yönetim sistemiyle, donanımla ve işletim sistemiyle ilişkisinde istenen sonuçları veriyor mu? Burada sistem hata yapmaya zorlanarak hataların hangi noktada ortaya çıktığı bulunur. Sistem testleri güvenlik, dayanıklılık ve performans açısından yapılır.

- **Güvenlik testi (security test):** Sistemin dışarıdan müdahalelere karşı korunabildiği test edilir. Bunun için saldırı simülasyonları yapılır.

- **Dayanıklılık testi (stres testi):** Sisteme miktar, sıklık (frekans) ve hacim olarak aşırı, anormal yükleme yapıldığında nasıl dayandığı belirlenir.
- **Performans testi:** Hazırlanan yazılımlarda cevap süresinin kısa olması istenir. Sistemin çalışma performansı test edilir. Hoca performans testinin birim testi sırasında da saydam kutu biçiminde yapılabileceğini ekledi.

Sistem testinde hedef hataları bulmaktır: güvenlik hatası var mı, performans hatası var mı, dayanıklılık açısından stres hatası var mı? Hoca bir dayanıklılık testi örneği verdi: Çok kullanıcılı bir sistemde erişim, sınırdaki erişim sayısı çerçevesinde test edilir. Sınırdaki erişim sayısına ulaşıldığında, bundan bir fazla ve bir eksik olduğunda sistem bunu karşılayabiliyor mu? Bu kadar çok kullanıcıyla çalışırken problem var mı?

8.4 Kabul (Onay) Testi

Onay testi, bütünleştirme testinden, hatta bazen sistem testinden sonra yapılır. Amaç, müşteri tarafındaki kullanıcıların beklentilerinin gerçekleşip gerçekleşmediğini, gereksinimlerde belirtilenlerin sağlanıp sağlanmadığını görmektir. Buna onay testi, kabul muayenesi ya da kabul testi (acceptance test) denir.

- **Girdileri:** bütünleşmiş yazılım, gereksinimler ve kullanıcı belgeleri. Müşteri yazılımı kabul eder: "Evet, benim istediğim gereksinimlere göre hazırlanmıştır."
- Yazılım onay testi sırasında yazılım gereksinim spesifikasyonu (SRS) bulunmalıdır. Yazılım SRS ile karşılaştırılarak bir dizi kara kutu testi uygulanır. Hoca, teslim sürecine yaklaştıkça daha çok kara kutu testi kullanıldığını vurguladı.
- Uygulama, önceden hazırlanmış test planına göre yürütülür. Müşteriyle birlikte, SRS belgesinde tanımlanan her şeyin yapılıp yapılmadığına bakılır. Kullanıcı belgeleri takip edilerek yazılım kullanıldığında, müşterinin başta belirttiği gereksinimlerin karşılanıp karşılanmadığı görülür.

Konfigürasyon gözden geçirmesi (configuration review): Bu aşamada yazılım konfigürasyonu (yazılım düzeni) da incelenir. Yazılım konfigürasyonu şunlardan oluşur:

- Kullanıcı belgeleri: kullanım sürecini anlatan belgeler.
- Tasarım belgeleri: yazılım sistemi tasarımıyla ilgili diyagramlar ve belgeler.
- Test belgeleri: hangi test verisiyle inceleme yapıldığı, hangi testlerin yapıldığı.
- Kayıtlar.

Teslim edilecek bütün belgeler ve istenen her şey hazırlanmış mı diye bakılır. Ardından yazılımın teslim süreci gelir.

Alfa ve beta testi: Kabul aşamasında alfa ve beta testleri yapılır.

	Alfa testi	Beta testi
Kim yapar?	Kullanıcıya benzer bir grup; proje yöneticisinin denetiminde, proje ekibinden bir grup	Yazılımın kullanıcıları ya da potansiyel kullanıcı olarak seçilmiş gruplar
Destek	Yazılım geliştirici grubun desteğiyle	Tamamen müşteri tarafında
Bakış	Kullanıcı gözüyle	Gerçek kullanıcılar

9. Test Tipleri: Saydam Kutu, Kara Kutu, Gri Kutu

Test adımlarından sonra hoca test tiplerine geçti. Test tipleri yalnız kara kutu ve saydam kutu değildir; üçüncü bir tip olarak gri kutu testi de vardır.

9.1 Saydam Kutu Testi

Saydam kutu testi doğrudan modül üzerinde uygulanır ve programın iç yapısını denetler. Test sırası açısından en çok birim testinde kullanılır. Test programıyla yazılımdaki koşul ve döngü

durumları çalıştırılarak mantıksal yollar sınanır. Programın tasarlandığı ve beklendiği yapıda olup olmadığı test edilir.

Programın tamamını bütün olası biçimlerde çalıştırıp test edemeyiz. Bu yüzden temel yollar bulunur. Tasarımın kontrol yapısı, yani algoritma kullanılarak, bir modüldeki bütün bağımsız yolların en az bir kez çalıştırılması garanti edilir. Bunun için bütün mantıksal kararların hem doğru hem yanlış durumunda gideceği yolları test edecek test verisi kullanılmalıdır.

Hoca İngilizcesi white box olan bu teste "beyaz kutu" yerine "saydam kutu" dediklerini belirtti. Sebebi, yazılımın iç yapısını görerek test ettiğimizin anlaşılmasını istemeleridir. Yalnızca dışarıdan görebiliyor, algoritmayı ve iç ayrıntıları göremiyorsak zorunlu olarak kara kutu testi yapılır.

Temel yol testi (basis path testing): Bir temel yol kümesi oluşturmaya dayanır. Önce McCabe ölçütüyle bir birimin, modülün ya da yazılım parçasının kaç temel yolu olduğu bulunur. Sonra bu temel yolların her birini çalıştıracak test verisi ve buna uygun test programları hazırlanır. Hedef, yazılım parçasının bütün temel yollarından geçen akışları test edecek test verisini hazırlamaktır.

Döngü testi: Bir girişi ve bir çıkışı olan bir yapı olarak soyutlanmış her döngü test edilir. Döngüde şu hatalar aranır:

- Başlama hatası.
- İlerleme hatası.
- Artma hatası.
- Sınırlama hatası.
- Sonsuz döngü hatası.

Test sonunda döngü yapısı geçerli olarak onaylanmış olur. Basit, iç içe ve birbirine bağlı bütün döngülerde başlangıç ve bitiş noktalarına, döngü tasarımına, iç içe döngülerin kurallarına uyulup uyulmadığına ve birbirini kesen yollar olup olmadığına bakılır.

Hoca dersin sonuna doğru McCabe ölçütünün ve temel yol sayısı hesabının bu derste işlenmediğini, yazılım ölçütleri konusunda görüleceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** McCabe'in ölçütü çevrimsel karmaşıklık (cyclomatic complexity) olarak bilinir. Akış grafında E kenar, N düğüm sayısı ise $V(G) = E - N + 2$ olur. Eşdeğer olarak, karar düğümü sayısı P ise $V(G) = P + 1$ yazılabilir. $V(G)$, temel yol kümesindeki bağımsız yol sayısını, yani temel yol testinde hazırlanması gereken en az test durumu sayısını verir.
>
> **Ek bilgi (derste anlatılmadı):** Döngü testinde yaygın sınıflama dört türdür: basit, iç içe (nested), bitişik (concatenated) ve yapısız (unstructured) döngüler. Altyazıdaki "birbirine bağlı" büyük olasılıkla bitişik döngüleri karşılar. Yapısız döngüler genelde test edilmeden önce yapısal biçime dönüştürülmesi önerilen döngülerdir.

9.2 Kara Kutu Testi

Kara kutu testi, iç yapı bilinmediğinde, kaynak kod ve iç yapı kullanılmadan yazılımı test etmek için kullanılır. Özellikle yazılımın fonksiyonel çalışmasına bakılır. Yazılım bütünleştirilirken yazılımın ara birimi üzerinde yapılan bir sınamadır; daha çok bütünleştirme testi ve onay testi sırasında kullanılır.

Kara kutu testiyle şunlar değerlendirilir: yazılım işlevleri yerine getiriliyor mu, girdiler kabul ediliyor mu, çıktılar doğru üretiliyor mu, dosyalardan okuma ve yazma gibi dış işlemlerin bütünlüğü sağlanıyor mu? Kara kutu testinde mantıksal iç yapıdan çok temel sistem modeli incelenir.

> **Hocanın vurgusu:** Bir yazılımı test ederken kara kutu ile saydam kutu arasında seçim yapmıyoruz. İkisi birlikte uygulanır. Böylece yazılım ara biriminin geçerliği onaylanır ve yazılımın işlerliği güvence altına alınır.

Kara kutu testinin bulduğu hatalar:

- **İşlev hataları:** Kullanıcının tarif ettiği gereksinimlerden farklılık var mı, tarif edilen işlemlerin dışında bir iş yapılmış mı? Hatalı ve eksik işlevler.

- **Ara birim hataları.**
- **Veri yapılarında ve veritabanı erişiminde hatalar.**
- **Performans hataları:** Bunlar sistem testi sırasında bulunur.
- **Başlatma ve sonlandırma hataları:** Döngü dışındaki işlemlerin ve veritabanı uygulamalarının başlangıç ve bitiş hataları.

Kara kutu testinde yazılım sistemi bir kara kutudur: iç yapısını bilmeyiz, dışarıdan test verisi göndeririz. Doğru verinin yanında anormal davranışa yol açacak hatalı veri de gönderilir. Girdilere göre sistemin nasıl çıktı ürettiğine, hatalı girdilerde nasıl çıktılar verdiğine bakılır. Ayrıca hataların kesintiye yol açmadan nasıl karşılanıp sistemin çalışmaya devam edebileceği belirlenir.

9.3 Gri Kutu Testi

Gri kutu testinde iki yaklaşım karıştırılır. Örneğin test verisi kara kutu testindeki gibi hazırlanır ama test saydam kutu gibi çalıştırılır. Tersi daha sık görülür: test verisi saydam kutu testindeki gibi hazırlanır, test kara kutu gibi çalıştırılır. İkisinin karışımı olan testlere gri kutu testi denir. Hoca, son yıllarda gri kutu testlerinin daha ağırlıklı yapıldığını ve bu yüzden üçüncü bir tip olarak daha sık karşımıza çıktığını söyledi.

10. Kara Kutu Testinde Test Verisi Üretmek

10.1 Test Durumu (Test Case) Nedir?

Hoca test durumunu (test case) test veri grubu olarak tanımladı. Saydam kutu testinde iç yapıyı bildiğimiz için, her temel yola dallanmayı sağlayacak test verisi oluştururuz. Bunların her biri bir test durumudur. Kara kutu testinde ise iç yapıyı bilmeyiz. Bu durumda test verisi iki yoldan oluşturulur: ya dış spesifikasyonlara ve gereksinimlere göre, ya da girdi uzayının belli aralıklara bölünmesiyle. Her iki durumda da birden fazla test durumu oluşturulur.

10.2 Gereksinimlere Göre Test Verisi

Gereksinimlere göre test durumu oluşturmak daha çok onay testinde, yani alfa ve beta testi aşamasında yer alır. Hocanın örneği bir öğrenci bilgi sistemiydi. Sistemde ders kayıt işlemi yapılacak. Farklı tipteki öğrenciler için ders seçiminin nasıl gerçekleştiği test edilir:

- Bölüm öğrencisi.
- Başka bölümden gelen öğrenci.
- Dışarıdan gelen öğrenci.

Bu farklı öğrenci tiplerinin girdi verisi verilerek, yani dış gereksinim spesifikasyonlarına göre test verisi oluşturulur.

Bütünleştirme testinde ise daha sayısal, aralıklar hâlinde çalışmak uygundur. Bunun başlıca yöntemleri şunlardır: eşdeğer bölümleme, sınır değer analizi ve neden-sonuç grafı çizimi. Hoca ilk ikisi üzerinde durdu.

10.3 Eşdeğer Bölümleme

Eşdeğer bölümlemede (equivalence partitioning) girdi uzayı sırayla eşdeğer bölümlere ayrılır. Programın girdi alanı, test programları oluşturulabilecek veri sınıflarına bölünür. Her test programı belirli bir sınıftaki hatayı ortaya çıkarmak üzere çalıştırılır. Bu bölümlerin her birinden bir kez test verisi gönderilir.

Hocanın örneği: 1 ile 100 arasında bir sayı girilecekse, şu gruplar tanımlanabilir:

- 1–25
- 26–56
- 57–85
- 85–100 (altyazıdaki biçimiyle)

Her gruptan bir test verisi gönderilir. Test verisi rastgele seçilseydi, seçilen değerler aynı aralığa düşebilirdi. Bölümleme sayesinde farklı alanlar test edilmiş olur.

> **Ders sonrası not:** Altyazıda son iki grup “57 ile 85” ve “85 ile 100” diye geçiyor; 85 iki gruba birden düşüyor. Eşdeğer sınıflar örtüşmemelidir; son grup 86–100 olmalıdır. Bu düzeltmeyle dört grup 1–100 aralığını boşluksuz ve örtüşmesiz kaplar (25 + 31 + 29 + 15 = 100 değer; python ile kontrol edildi). Ayrıca eşdeğer bölümlemede geçersiz sınıflar da test edilir: 1’den küçük ve 100’den büyük değerler. Örnekteki aralık sınırları keyfîdir; gerçek uygulamada bölümler spesifikasyonda farklı davranış beklenen bölgelere göre çıkarılır.

10.4 Sınır Değer Analizi

Sınır değer analizi (boundary value analysis) de bir kara kutu testidir; iç yapı bilinmez. Test verisi şu mantıkla seçilir: Yazılımda bir hata varsa sınırlarda ortaya çıkar. Veri yapılarının, alt program parametrelerinin sınırları ve uç değerleri seçilir. Genellikle en büyük, en küçük ya da en az beklenen değerler alınır. Hata varsa bunlarda vardır; bunlarda hata yoksa diğerlerinde de hata olmadığı kabul edilir.

> **Hocanın vurgusu:** Sınır değer analizinin mantığı: Hata varsa çoğunlukla sınırlarda ortaya çıkar.

Dayanıklılık (stres) testleri genellikle bu mantıkla, sınır değer analizi biçiminde yapılır. Hocanın örneği: Sistemi aynı anda 200 kullanıcı kullanacaksa, 199 kişi bağlandığında sistemin nasıl davranacağını simüle eden bir stres testi oluşturulur.

> **Ders sonrası not:** Burada yalnız 199 değeri anıldı. Sınır değer analizinde sınırın kendisi (200), bir altı (199) ve bir üstü (201) birlikte denenir. Hoca sistem testini anlatırken de sınırdaki erişim sayısının, bir eksiğinin ve bir fazlasının denenmesi gerektiğini söylemişti.

10.5 Neden-Sonuç Grafı

Neden-sonuç grafı (cause-effect graph) yöntemlerden biri olarak adıyla anıldı, ayrıntısına girilmedi.

> **Ek bilgi (derste anlatılmadı):** Neden-sonuç grafında girdi koşulları (nedenler) ile çıktı davranışları (sonuçlar) VE, VEYA, DEĞİL gibi mantıksal bağlantılarla bir grafa dökülür. Bu graf bir karar tablosuna çevrilir ve tablonun her sütunu bir test durumu olur. Yöntem, birden fazla girdi koşulunun birlikte etkisini sistemli biçimde test etmeyi sağlar.

11. Test Otomasyonu

Hoca sınır değer analizinin stres testlerinde kullanıldığını düşünürsek bu tür testlerde daha çok test otomasyonu kullanılacağını ekledi. Test otomasyonu olmadan, 199 kullanıcının aynı anda işlem yaptığı durumdaki performansı gerçek hayatta denemek zordur. Bu işi simülasyonla yapacak bir test otomasyonu kurmak gerekir. Hocanın ifadesiyle, yazılım analizi ve tasarımı sırasında araçlar kullandığımız gibi, test sırasında da test araçlarını kullanmak durumunda kalabiliriz.

12. Doğrulama ve Geçerleme

Doğrulama ve geçerleme (verification and validation, V&V) konusu, hocanın test adımlarını sorduğu bir öğrencinin cevabında “validasyon” kelimesini kullanmasıyla açıldı. Hoca bu konudan notlarda söz etmediğini, öğrencinin hatırlatmasının iyi olduğunu söyledi.

İki terim de Türkçede doğrulama ya da geçerleme diye çevrilebilir, ama birine doğrulama, öbürüne geçerleme denmesi seçilmiştir:

- **Doğrulama (verification):** Yaptığımız işin doğrulanmasıdır. İşi doğru yolla mı yapıyoruz? Birim testinde hazırlanan algoritma adımları doğru mu, bütünleştirme testinde modüller doğru birleştirilmiş mi diye bakılır.
- **Geçerleme (validation):** Sonuçta doğru işi, doğru sonucu elde ediyor muyuz? Elde ettiğimiz yazılım müşterinin istediği yazılım mı?

Her test adımında doğrulama ve geçerleme farklı ağırlıklarla yapılır:

Test adımı	Doğrulama	Geçerleme
Birim testi	Var	—
Bütünleştirme testi	Var	Var (alınan sonuçlar da doğru mu?)
Sistem testi	Var	Ağırlıklı
Kullanıcı (kabul) testi	—	Tamamen

Hoca test stratejisini burada bir kez daha toparladı. Önce kodun en küçük birimleri test edilir. Sonra bu birimler birleştirildiğinde doğru olup olmadığı test edilir (bütünleştirme testi). Sonra bütün olarak çalışan sistemin doğru çalışıp çalışmadığı test edilir (sistem testi). En sonda kullanıcı gereksinimlerine, SRS'e uygun olup olmadığı test edilir (kullanıcı testi). Bütün bu adımlara test süreci adımları ya da test stratejisi denir.

13. Test ve Hata Ayıklama

Hoca dersin kapanışında temel ilkeyi bir kez daha söyledi.

> **Hocanın vurgusu:** Testteki tek amaç hatayı bulmaktır. Hatayı düzeltmek testin dışındadır.

Hata düzeltme işlemi hata ayıklamadır (debugging): testte bulunan hataların ayıklanması ve düzeltilmesi. Bir öğrencinin saydam kutu sorusunda "debugging" cevabını vermesi üzerine de hoca, hata ayıklamanın test tipi olmadığını, hataları düzeltme işlemi olduğunu belirtti.

Örnek Sorular

1. (derste sözlü, 00:14:36) Tasarım konusunda gördüğümüz kavramlardan, tekrar kullanılabilirliği (reusability) sağlayan en önemlileri hangileridir?

> **Çözüm (derste verildi):** Nesneye yönelik yaklaşım; az bağımlılık ve modülerlik (sınıflar arasında az bağımlılık); tasarım kalıplarının (design patterns) kullanılması; yazılım kütüphanelerinin kullanılması. Bunlar daha az iş gücü harcanmasını sağlar ve test edilmiş parçalar kullanıldığı için testi kolaylaştırır.

2. (derste sözlü, 01:27:50) Yazılım geliştirme sürecindeyiz. Bir test stratejisinden bahsettik. Bu sırada hangi testleri yaparız, test adımları nelerdir?

> **Çözüm (derste verildi):** Alttan itibaren birim testi, bütünleştirme testi, sistem testi ve kullanıcı (kabul) testi. Önce kodun en küçük birimleri, sonra birleştirilmiş birimler, sonra bütün sistem, en son SRS'e uygunluk test edilir. Her adımda farklı ağırlıklarla doğrulama ve geçerleme yapılır.

3. (derste sözlü, 01:33:21) Test tipleri nelerdir? (Test adımları değil; testi nasıl yaptığımız.)

> **Çözüm (derste verildi):** Saydam kutu (white box), kara kutu (black box) ve gri kutu testi. Saydam kutuda programın içeriği bilinir ve kontrol edilir. Kara kutuda kod olmadan, dışarıdan bakılarak yazılımın istenen işi yapıp yapmadığı ya da dayanıklılığı test edilir. Gri kutuda test verisi bir yaklaşımla hazırlanır, test öbür yaklaşımla çalıştırılır.

4. (derste sözlü, 01:37:08) Saydam kutu testini hangi tiplerde yapabiliriz? İç yapıyı görüyoruz, ama iç yapının nelerine bakarız?

> **Çözüm (derste verildi):** Program yapılarına göre iki tip görüldü: temel yol testi (basis path testing), yani kontrol yapıları çerçevesinde her program akışının doğru yürüyüp yürümediği; ve döngü testi, yani programdaki döngülerin ve artımların doğruluğu. Öğrencinin verdiği "debugging" cevabı yanlıştır; hata ayıklama bir test tipi değil, hataları düzeltme işlemidir.

5. (derste sözlü, 01:39:48) Kara kutu testinde test tipleri nelerdir? Test verisini dışarıdan nasıl veririz?

Çözüm (derste verildi): Test verisini oluşturma yoluna göre: eşdeğer bölümleme (girdi uzayını bölümlere ayırıp her bölümden bir test verisi göndermek), sınır değer analizi (en büyük ve en küçük değerleri seçmek) ve gereksinimlere/işlevlere göre test durumu seçmek (örneğin öğrenci bilgi sisteminde ders kayıt modülüne farklı tipte öğrenci bilgisi göndermek).

6. **(derste sözlü, 01:43:36)** Testin amacı hatayı bulmaksa, hata düzeltme işlemi nedir?

   Çözüm (derste verildi): Hata ayıklama (debug) işlemidir: testte bulunan hataların ayıklanması ve düzeltilmesi.

Dersin Toparlaması

Bu derste yazılım kalitesi ve test konusu işlendi. Kaliteli yazılımın özellikleri (doğruluk, güvenilirlik, verimlilik, güvenlik, kullanışlılık, bakım kolaylığı, taşınabilirlik, yeniden kullanılabilirlik, birlikte çalışabilirlik) tanımlandı. McCall modelinde bu faktörlerin doğrudan değil, metriklerin ağırlıklı toplamıyla tahmin edildiği görüldü. Kaliteyi sağlamanın temel ilkesi, hataları olabildiğince erken bulmaktır, çünkü hata bulmanın maliyeti süreç ilerledikçe artar. Test stratejisi, geliştirme sürecinin tersine birim, bütünleştirme, sistem ve kabul testleri olarak ilerler. Test tipleri saydam kutu (temel yol ve döngü testi), kara kutu (gereksinimlere göre, eşdeğer bölümleme, sınır değer analizi) ve gri kutudur; saydam ve kara kutu birlikte uygulanır. Doğrulama işin doğru yapılıp yapılmadığını, geçerleme doğru işin yapılıp yapılmadığını sorar. Testin tek amacı hatayı bulmaktır, düzeltmek hata ayıklamanın işidir.

Hoca, kalan derste yazılım proje yönetimi ve yazılım ölçümünün işleneceğini söyledi. Bu derste adı geçen kalite metrikleri ve McCabe ölçütüyle temel yol sayısının hesabı o konuda pekiştirilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.